Exploring Recurrent Architectures for EMG-to-QWERTY Decoding

Akash Madiraju
University of California, Los Angeles
akashm153@ucla.edu

Justin Chen
University of California, Los Angeles
justinc918@ucla.edu

Prabhvir Singh
University of California, Los Angeles
prabab@ucla.edu

Tyler Xiao
University of California, Los Angeles
tylerxiao@ucla.edu

Abstract

The emg2qwerty dataset is a collection of surface electromyography (sEMG) data corresponding to keystrokes on a standard keyboard. In this paper, we explore using different model architectures to best fit this data. The baseline Time-Depth Separable (TDS) model, a convolutional neural network, performs adequately, but we believe performance can be improved through the use of recurrent neural networks (RNN). In all, we consider a vanilla RNN (an RNN with no additions), an RNN with Gated Recurrent Units (GRU), and a hybrid model consisting of the base TDS model that leads into an RNN with GRU. We also introduce data augmentation techniques including resampling the data at a different frequency and dropout for channels. Our results reveal that the vanilla RNN model does not outperform the baseline model, but RNN with GRU does slightly better. The hybrid model shows the best performance, combining the strengths of both the baseline model and the RNN with GRU model. We also demonstrate that the data augmentation techniques we introduce improve generalization for the models we considered.

1 Introduction

The central goal of this project is to identify and train a model that achieves lower loss and better character error rate (CER) than the base TDS model under the final project guidelines. Because of limited computing resources, all of our experiments were conducted on a single subject, user 89335547. We also kept our models relatively small, targeting fewer than 5 million parameters, both to reduce the risk of overtraining on a limited dataset and to allow faster iteration within the project timeline.

This setting naturally motivated us to explore recurrent architectures. The emg2qwerty task maps continuous wrist sEMG signals to an unsegmented character sequence, so the temporal structure of the signal is central to successful decoding. In contrast to the baseline TDS model, which is primarily convolutional, recurrent neural networks (RNNs) offer a direct mechanism for processing information over time. We therefore focused first on a vanilla RNN as a lightweight temporal baseline, then considered gated recurrent variants such as gated recurrent units (GRUs) and long short-term memory (LSTM) networks to improve optimization and temporal credit assignment on limited data. We also considered a hybrid CNN+GRU architecture as a way to combine spatial feature extraction with temporal modeling if purely recurrent models proved insufficient.

In addition to architecture choice, we considered data augmentation and input reduction strategies to improve generalization without substantially increasing model size. In particular, we explored whether reducing signal detail through lower-frequency resampling or restricting the model to subsets

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Table 1: Official single-user split used for the main experiments

Split	Sessions	User ID	Notes
Train	16	89335547	Official training sessions from the provided config
Validation	1	89335547	Used for model selection and early comparison
Test	1	89335547	Held out for the final architecture comparison

Table 2: Model families used in the architecture comparison

Model	Encoder configuration	MLP features	Dropout
TDS baseline	4 TDS convolution blocks, kernel width 32	[384]	–
RNN	2-layer bidirectional RNN, hidden size 256	[256]	0.0
GRU	4-layer bidirectional GRU, hidden size 128	[256, 256]	0.1
Hybrid CNN+GRU	4 TDS blocks + 4-layer bidirectional GRU, hidden size 192	[192, 192]	0.1

of available channels could encourage the models to rely on more robust patterns rather than subject-specific noise. Together, these design choices frame the main question of this report: which compact recurrent or hybrid architecture most effectively improves decoding quality over the base TDS model on the provided single-user split?

2 Methods

2.1 Dataset and evaluation setup

This project uses the emg2qwerty dataset released by Meta Reality Labs [2]. For the main comparison, we follow the official single-user split provided by the course codebase repository [4] for user 89335547. All four model families are trained and evaluated on the same 16/1/1 train/validation/test session split so that differences in performance can be attributed primarily to encoder design rather than to changes in data availability or subject identity.

We evaluate all models primarily with character error rate (CER), which is the main metric emphasized in the project handout and is well matched to the CTC decoding objective. We also track insertion, deletion, and substitution error rates to better diagnose failure modes, since different architectures may over-predict characters, miss characters entirely, or confuse similar temporal patterns in different ways. Validation CER is used for model selection, while the final held-out test session is reserved for the architecture comparison reported in the Results section.

2.2 Model families

Our architecture study compares four encoder families that share the same windowed EMG front end and CTC decoding pipeline. The provided TDSConv model serves as the course baseline and offers a strong convolutional reference point for the task. We then compare three alternatives developed around explicit temporal recurrence: a vanilla bidirectional RNN as the simplest recurrent baseline, a deeper bidirectional GRU to test whether gating improves stability and sequence modeling, and a hybrid CNN+GRU model that applies TDS-style convolutional blocks before a recurrent encoder. This comparison is designed to test whether temporal recurrence alone is sufficient to outperform the baseline or whether a mixed convolutional-recurrent encoder yields a better trade-off on the limited single-user setting.

All four models operate on the same log-spectrogram EMG representation and produce framewise character logits for greedy CTC decoding. This shared output structure lets us compare the effect of encoder choice without changing the loss, decoder, or evaluation metric between runs.

2.3 Vanilla RNN

We first picked an RNN with no modifications to fit the data. An RNN is a neural network with the ability to retain information in the short-term from past data using a feedback loop. Because of this, we believed that this model has potential to fit sEMG data well, given that sEMG data has a time dimension. Being the most simple form of RNN, we also used this model as a starting point for further improvements we anticipate needing. These improvements include something to address the fact that RNNs often struggle with vanishing gradients. Gradients sometimes vanish, or become too small and therefore negligible, when being passed through too many layers. If the input data is long enough (temporally), this could be an issue, as the RNN would slowly lose earlier data. We used a bidirectional RNN to allow the model to have a memory of both past and future data. We also used dropout to prevent overfitting. It was after inadequate empirical performance that we considered a more advanced RNN model.

2.4 RNN with GRU

After the disappointing performance of RNN, we investigated using an RNN with Gated Recurrent Units (GRU) [5]. We chose GRU over LSTM due to GRU requiring less parameters, with us moving on to LSTM in case GRU still did not perform adequately. In theory, RNN with GRU will have better performance than a vanilla RNN due to the long length of the sEMG data. We suspect part of the performance issues with a vanilla RNN is from vanishing gradients, an issue GRU does not suffer from as much.

2.5 Hybrid TDS with GRU

To begin, we noticed that the baseline model performed decently. However, it was hampered by its inability to process temporal information, the way an RNN can. From here, we thought of combining the base TDS model with the best RNN model we had at the time, which was an RNN with GRU. Similar hybrid CNN+GRU designs have also been reported in related sequence-modeling settings [6]. The idea here is that the initial convolutional layer could extract spatial features from the sEMG data, while the recurrent layer could process temporal relationships between said spatial features.

We started with the baseline parameters for the TDS, and the best parameters for GRU. This included the parameters for dropout (0.1) and bidirectional flow (True).

2.6 Data Augmentation

The baseline model already makes use of some data augmentation. These include Temporal Alignment Jitter, Random Band Rotation, and SpecAugment. All of these techniques introduce noise into the sEMG data, which helps with generalization. Quick ablation testing using the baseline model shows that these techniques do indeed help with accuracy. For all of our trials, we keep these baseline techniques.

New techniques that build off from the baseline include resampling the data at a lower frequency than the baseline 2000 Hz and deactivating some of the channels. We considered resampling the data because this approach would have a smoothing effect across the sEMG data. In theory, resampling at a lower frequency would remove many of the outliers present in sEMG data, potentially improving model performance.

We also considered disabling a subset of the training data, but theorized that this would not improve training. We were not training on much data to begin with. Our models typically struggled with overfitting the training data, and in this situation, we should look towards strategies to either find more data or generalize instead of removing data. Empirically, this resulted in similar results but multiple times more epochs to train compared to the base case. We did not try this technique again.

2.7 Electrode Channel Selection Experiments

In addition to architectural comparisons, we evaluated how the number and spatial distribution of EMG electrode channels affect decoding performance. The default configuration uses all 16 channels available in the dataset. To study redundancy between neighboring electrodes, we trained the hybrid CNN+GRU model using several reduced channel subsets. The subsets tested were the first half of

Table 3: Training configurations for the compared model families

Model	LR	Batch size	Epoch budget	Scheduler	Notes
TDS baseline	1e-3	32	40	Warmup + cosine	Original course baseline
RNN	3e-4	32	100	Cosine annealing	2-layer bidirectional recurrent encoder
GRU	1e-4	32	40	Cosine annealing	4-layer gated recurrent encoder
Hybrid CNN+GRU	3e-4	32	40	Cosine annealing	TDS front end + bidirectional GRU encoder

Table 4: Architecture comparison with validation and test metrics

Model	Epochs	Val loss	Val CER	Val DER	Val IER	Val SER	Test loss	Test CER	Test DER	Test IER	Test SER
RNN	100	1.463	48.892	11.099	15.175	22.619	1.191	39.83	1.794	18.695	19.34
GRU	40	0.626	19.67	2.99	4.36	12.32	0.625	20.60	3.24	2.46	14.89
Hybrid CNN+GRU	40	0.531	16.22	1.68	3.17	11.36	0.511	16.90	1.73	2.49	12.69
TDS baseline	40	0.757	23.17	2.02	7.53	13.6	0.798	24.34	1.84	6.09	16.40

channels [0–7], the second half [8–15], an evenly spaced subset of even channels [0,2,4,6,8,10,12,14], and a smaller evenly spaced subset [0,4,8,12]. All other preprocessing, training hyperparameters, and model architecture settings were kept fixed so that differences in performance could be attributed primarily to the choice of input channels.

2.8 Training protocol and hyperparameters

Across experiments, we keep the data pipeline fixed and vary only the encoder family and its associated hyperparameters. Input windows have length 8000 with left-context/right-context padding of 1800 and 200 samples, respectively. Raw left and right EMG streams are converted to tensors and transformed into log-spectrogram features using $n\_fft = 64$ and hop length 16. During training, we apply the augmentations defined in the codebase, including random band rotation, temporal alignment jitter, and SpecAugment. Validation and test runs use the deterministic evaluation pipeline without these stochastic augmentations.

Optimization follows the PyTorch Lightning and Hydra configuration scaffold used throughout the repository. Unless otherwise noted, runs use batch size 32, four dataloader workers, Adam optimization, and checkpoint selection based on validation CER. Each model uses the CTC greedy decoder for evaluation, which keeps decoding consistent across the convolutional, recurrent, and hybrid architectures. Exact CER and loss values are deferred to the Results section, but the main training settings for the compared models are summarized in Table 3.

In addition to the main architecture comparison, the repository also supports exploratory changes such as reduced channel subsets and related preprocessing variations. Those experiments are treated as secondary analyses rather than the core comparison in this report, so we describe them separately in the Results section only when they contribute directly to the final model discussion.

3 Results

Table 4 summarizes the validation and test performance of the four model families considered in this study. For fairness, all models were evaluated on the same official single-user split. The vanilla RNN was trained for 100 epochs because its validation CER improved more slowly, whereas the GRU, hybrid CNN+GRU, and TDS baseline all reached stable performance within the 40-epoch budget.

3.1 Channel Subset Study

We next evaluated how reducing the number of EMG channels affects decoding performance. Table 5 summarizes the results of training the hybrid CNN+GRU model with several different channel subsets.

Table 5: Channel subset comparison using the hybrid CNN+GRU model

Channel subset	Number of channels	Test CER
All channels	16	16.90
First half: 0–7	8	22.93
Second half: 8–15	8	25.01
Even channels: 0,2,. . . ,12,14	8	17.55
0,4,8,12	4	23.71

The full 16-channel configuration achieved the best overall performance. However, the subset containing all even-numbered channels [0,2,4,6,8,10,12,14] performed nearly as well despite using only half as many electrodes. In contrast, subsets that used only the first half [0–7] or the second half [8–15] of the channels performed worse. Additionally, the subset containing only the multiple of 4 channels [0,4,8,12] did about as well as the subsets using only the first half or the second half of the electrodes, again despite using only half as many electrodes.

4 Discussion

We now discuss the performance of our different models and the results of our experiments.

GRU vs. Vanilla RNN. Vanilla recurrent neural networks struggled to achieve competitive performance on the EMG-to-QWERTY task. One likely reason is the well-known vanishing gradient problem in simple RNN architectures. EMG typing sequences are relatively long, and the model must preserve information across many time steps in order to correctly associate patterns of muscle activation with individual characters. In a vanilla RNN, gradients propagated through many recurrent steps can shrink rapidly, making it difficult for the model to learn long-range temporal dependencies.

The GRU model significantly improved performance over the vanilla RNN. This is consistent with prior empirical work showing that gated recurrent units often outperform more traditional recurrent units on sequence modeling tasks [5]. GRUs incorporate gating mechanisms that regulate how information is updated and retained in the hidden state. In particular, the update and reset gates allow the network to preserve useful temporal context while discarding irrelevant information. This improves gradient flow during training and allows the model to capture longer temporal relationships in the EMG signal. Because typing gestures unfold over time and often involve subtle transitions between muscle activations, the ability to maintain relevant temporal information is particularly important for this task.

An interesting observation is that GRU's improved performance mainly stems from a much-improved insertion error rate. One possible explanation for this improvement is that the gating mechanisms in GRUs help stabilize the hidden state over time. In CTC-based sequence models, insertion errors often occur when the model produces spurious character predictions across consecutive frames. EMG signals can contain short-lived fluctuations due to noise or minor muscle activations, which a simple RNN may interpret as new characters. The update and reset gates in GRUs allow the model to retain relevant temporal context while filtering out transient variations in the signal. As a result, the GRU model is less likely to generate unnecessary character predictions, which can lead to a reduction in insertion errors.

Benefits of Combining Convolution and Recurrence. While the GRU model improved performance relative to the vanilla RNN, the hybrid CNN+GRU architecture achieved the best results overall. This aligns with related findings that CNN+GRU hybrids can benefit from combining convolutional feature extraction with recurrent sequence modeling [6]. One possible explanation is that convolutional layers are well suited for extracting local structure from EMG signals before temporal modeling is applied. EMG data contains localized patterns in both time and frequency that correspond to muscle activations associated with specific finger movements. The TDS convolutional blocks can learn filters that detect these local activation patterns and produce a higher-level representation of the signal.

Once these local features are extracted, the recurrent GRU layers can focus on modeling the temporal relationships between them. This division of labor allows the network to separate two difficult tasks:

feature extraction and sequence modeling. In contrast, a purely recurrent model must learn both simultaneously, which may make optimization more difficult when training data is limited.

The hybrid architecture therefore benefits from combining the strengths of both approaches. Convolutional layers efficiently capture local spatial and spectral structure in the EMG signals, while recurrent layers model the longer temporal dependencies associated with typing sequences. This combination likely explains why the hybrid model achieved the lowest character error rate among the tested architectures.

Sensor Redundancy and Channel Efficiency. The channel subset experiments also provide insight into how spatial information from EMG sensors affects decoding. While the model using all channels performed best, the evenly spaced subset of even-numbered channels achieved nearly identical performance despite using only half as many electrodes (17.55 CER vs. 16.90 CER). In contrast, subsets that concentrated sensors within a single contiguous region of the electrode array performed substantially worse, with the first-half [0–7] and second-half [8–15] configurations yielding 22.93 and 25.01 CER respectively (Table 5). Interestingly, a much smaller subset containing only four evenly spaced channels [0,4,8,12] achieved 23.71 CER, which is comparable to the performance of the larger contiguous 8-channel subsets. This suggests that neighboring electrodes may contain partially redundant information because EMG signals are spatially correlated across nearby sensors. As a result, maintaining spatial coverage across the forearm appears more important than simply increasing the number of adjacent electrodes. These results indicate that a reduced but well-distributed sensor configuration may retain most of the useful signal information while significantly lowering hardware complexity.

5 Future Work

We had set out to discover a model that could fit the given data for this subject significantly better than the baseline TDS model, and we found that in a TDS-GRU hybrid model. However, due to time and budget constraints, there are some choices of models we considered but did not have the resources to test. These include RNN with LSTM, another type of RNN that can avoid vanishing gradients. LSTM is generally better than GRU at tracking long-term information, so in future projects, we can explore how LSTM compares with GRU.

In addition, we considered using a transformer. A transformer takes in the entire data at once and decides which parts are important using attention, compared to CNNs and RNNs that rely on processing the data sequentially. This fundamental difference means that a transformer's performance could be drastically different from anything we have compared.

Finally, it can be interesting to train a hybrid model that uses an RNN without GRU. We can test whether or not a lack of long-term memory in the RNN can be made up for with spatial reasoning. While we do not expect the performance to exceed that of our current hybrid model, it can be interesting to compare to the baseline, given that a vanilla RNN had the worst performance (and therefore most room for improvement).

Limitations. This project currently focuses on a single user because of the compute required to train a model on the full dataset. However, that choice limits how strongly the results can generalize across users, sessions, and hardware conditions. In particular, performance differences observed on user 89335547 may reflect user-specific signal characteristics rather than trends that would hold across a broader population.

A second limitation is the constrained experimental budget. Although we compared multiple encoder families and performed targeted follow-up studies, the total number of full training runs remained limited by available time and GPU resources. As a result, the hyperparameter search was relatively narrow and was not exhaustive across all possible hidden sizes, layer counts, dropout settings, learning rates, or preprocessing choices.

We also did not run multiple random seeds for each final configuration. The reported CER values therefore correspond to single training runs rather than mean performance with variance estimates. This makes the comparison useful for a course project setting, but it also means that some of the reported gaps between models could partly reflect run-to-run randomness in optimization.

Table 6: Compute and reproducibility information

Framework	PyTorch Lightning	1.8.6
Main device	1×GPU	Tesla T4, 16 GB of VRAM, NVIDIA
Training time	∼2 hr / 40 epochs	
Dataset split	Single user 89335547	/model/data

Finally, our evaluation relies primarily on held-out performance from one test session and does not include broader robustness checks such as cross-user transfer, repeated-session averaging, or hardware variation. These omissions mean that the report should be interpreted as a focused single-user architecture study rather than a definitive conclusion about the best emg2qwerty model in general.

Reproducibility notes. Below are the exact training commands, environment, and best checkpoint settings for reproducing our results from the project repository [4].

RNN (main): `python-memg2qwerty.trainuser=single_usermodel=rnn_ctccluster=basictrainer.accelerator=gputrainer.devices=1trainer.max_epochs=40`

GRU (main): `python-memg2qwerty.trainuser=single_usermodel=gru_ctctrainer.accelerator=gputrainer.devices=1trainer.max_epochs=1trainer.max_epochs=40`

CNN+GRU (main): `python-memg2qwerty.trainuser=single_usermodel=hybrid_ctctrainer.accelerator=gputrainer.devices=1trainer.max_epochs=1trainer.max_epochs=40`

Channel selection (branch `hybrid-channel-selection`): `python-memg2qwerty.trainmodel=hybrid_ctcuser=single_usertrainer.accelerator=gputrainer.devices=1channel_subset.channel_indices=[0,1,2,3,4,5,6,7]`

Resampling: in `log_spectrogram.yaml`, add `resample:{_target_:emg2qwerty.transforms.Resample,orig_freq:old_frequency,new_freq:new_frequency}` and include `${resample}` in the transforms list.

Acknowledgments and Disclosure of Funding

This template follows the official NeurIPS 2024 style requested by the course handout and is adapted for the ECE C147/C247 final project on the emg2qwerty dataset. We would like to thank the original authors of the emg2qwerty dataset for providing the dataset and code for the initial model. We would also like to thank Professor Jonathan Kao and the ECE C147A/247A staff for providing clear course guidelines, mentorship, resources, and Google Cloud credits for our experiments.

References

[1] Alex Graves, Santiago Fernandez, Faustino Gomez, and Jurgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[2] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R. Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I. Mandel. *emg2qwerty: A large dataset with baselines for touch typing using surface electromyography*, 2024. arXiv:2410.20081.

[3] ECE C147/C247 staff. *ECE C147/C247, Winter 2026 project guidelines*, 2026.

[4] lordboba. *ece-c147a-final*. GitHub repository, 2026. Available at `https://github.com/lordboba/ece-c147a-final`.

[5] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. *Empirical evaluation of gated recurrent neural networks on sequence modeling*, 2014. arXiv:1412.3555.

[6] Almustapha A. Wakili, Babajide J. Asaju, and Woosub Jung. *Evaluating BiLSTM and CNN+GRU approaches for human activity recognition using WiFi CSI data*, 2025. arXiv:2506.11165.